Industrial Metal Stencil OCR API

REST API Specification & Frontend Integration Guide (Gemini Vision AI Engine)

Production Base URL: https://s-ocr.vercel.app	**CORS Policy:** Access-Control-Allow-Origin: *
Local Dev Base URL: http://localhost:5000	**Engine:** Gemini Vision AI (4x Round-Robin)

1. System Overview

This cloud vision service extracts standardized laser-cut stencil codes, stamps, and serial numbers from metal parts, structural beams, and agricultural implements. The backend uses Google Gemini Vision models (gemini-3.5-flash / gemini-3.8-flash) backed by **4 round-robin API key slots** with automatic error/quota failover, ensuring 100% resilience against rate-limiting, extreme low-light shadows, specular glare, inverted/upside-down (180°) orientations, and mirrored through-cuts.

2. Endpoints Summary

Method	Endpoint	Content-Type	Purpose
POST	/api/ocr	multipart/form-data	Upload camera/photo image & extract implement code
GET	/api/health	application/json	Heartbeat check & AI round-robin slot status
GET	/api/catalog	application/json	Fetch authorized implement models & state codes

3. Main Scan Endpoint: POST /api/ocr

Uploads an image (from smartphone camera or file picker) and returns the verified stencil code.

Field	Type	Required	Description
file / image	Binary File	Yes	Image file from camera/picker (JPEG, PNG, WEBP).
include_images	Boolean	No	Returns digital stamp & binary cutout in Base64.

Success Response Format (200 OK):

```
{
  "status": "success",
  "final_code": "GJ/142/446/2026/11881/UP",
  "parsed_info": {
    "full_code": "GJ/142/446/2026/11881/UP",
    "state_code": "GJ", "state_name": "Gujarat",
    "mfr_code": "142", "model_code": "446",
    "model_info": { "name": "TUSKER / SEMI HSS", "series": "Heavy Duty" },
    "year": "2026", "serial_no": "11881", "suffix": "UP",
    "is_valid": true, "orientation": "180deg upside down", "confidence": 0.99
  },
  "key_used_slot": 1, "model_used": "gemini-3.5-flash",
  "digital_stamp_base64": "...",
  "elapsed_ms": 3140.2
}
```

4. Secondary Endpoints & Reference Data

4.1 Health Check: GET /api/health

Call on app launch to verify backend availability and active Gemini key slots:

```
{
  "status": "healthy",
  "service": "Laser Cut Metal Stencil OCR API",
  "ai_engine": "Google Gemini Vision AI",
  "round_robin_slots": 4,
  "supported_models": ["444", "445", "446", "447", "3528", "3674"],
  "version": "2.0.0"
}
```

4.2 Catalog Reference: GET /api/catalog

Returns model categories (Regular, Heavy Duty, Brackets) and Indian state mappings:

```
{
  "status": "success",
  "catalog": {
    "444": { "name": "REG SMART / REG", "series": "Regular Series" },
    "445": { "name": "REG V1 / SEMI MSS", "series": "Regular / Semi Series" },
    "446": { "name": "TUSKER / SEMI HSS", "series": "Heavy Duty / Semi Series" },
    "3674": { "name": "BRACKET / STRUCTURAL MOUNT", "series": "Sub-Assembly" }
  },
  "states": { "GJ": "Gujarat", "MH": "Maharashtra", "MP": "Madhya Pradesh", "RJ": "Rajasthan", ... }
}
```

5. Frontend User Workflow (Camera ➔ AI ➔ Readout)

The recommended UX integration steps for the web/mobile frontend developer:

Step 1: Capture Photo: Use a camera input button (<input type='file' accept='image/*' capture='environment'>).

Step 2: Dispatch Request: Construct FormData, append file, and send a POST request to https://s-ocr.vercel.app/api/ocr.

Step 3: Loading State: Display an animated progress bar or pulsing laser scanner animation while AI processes (avg. 2-4 seconds).

Step 4: Primary Card: Render response.final_code prominently in bold monospace font (White text on Pure Black card for high factory contrast).

Step 5: Breakdown Chips: Display separate badge pills for State, Manufacturer Code, Model, Production Year, Serial Number, and Suffix.

Step 6: Digital Stamp: Bind response.digital_stamp_base64 directly to for immediate visual confirmation.

6. Production Frontend Integration Code

6.1 Production JavaScript / Fetch API Snippet:

```
async function scanMetalStencil(fileObject) {
  const formData = new FormData();
  formData.append("file", fileObject); // Camera capture or file picker File object
  formData.append("include_images", "true");

  try {
    const response = await fetch("https://s-ocr.vercel.app/api/ocr", {
      method: "POST",
      body: formData, // Browser sets multipart boundary automatically
    });

    if (!response.ok) {
      const err = await response.json();
      throw new Error(err.error || `HTTP ${response.status}`);
    }

    const data = await response.json();
    console.log("Scanned Code:", data.final_code); // e.g. "GJ/142/446/2026/11881/UP"
    console.log("Model:", data.parsed_info.model_code, "| Serial:", data.parsed_info.serial_no);
    return data;
  } catch (error) {
    console.error("OCR Scan Failure:", error);
    throw error;
  }
}
```

6.2 cURL / Postman Command:

```
curl -X POST "https://s-ocr.vercel.app/api/ocr" \
  -F "file=@/path/to/part_photo.jpg" \
  -F "include_images=true"
```

7. HTTP Status & Error Handling

Status	Meaning	Action Required
200 OK	Success. Code extracted.	Render final_code and metadata on screen.
400 Bad Request	No image file provided.	Ensure file is attached to key file or image.
502 Bad Gateway	AI Vision timeout.	Prompt user to retake photo with clearer lighting.
500 Server Error	Unexpected server issue.	Retry request; backend auto-rotates to next key slot.